

Programming

9_git

- [Книга Pro Git](#)
- [Документация Git](#)
- [Документация GitHub](#)



Кто такой Git?

Git (произносится «гит») — распределённая система управления версиями. Проект был создан Линусом Торвальдсом для управления разработкой ядра Linux, первая версия выпущена 7 апреля 2005 года

Простыми словами Git - это такая программа, которая позволяет просматривать историю изменений файлов с кодом, а также хранить эти самые файлы с кодом таким образом, что мы можем в любой момент откатиться до нужной версии.

На сегодня Git самый популярный инструмент для хранения и управления версиями кода. Есть у него и аналоги, но на них мы даже и не будем тратить время, т.к. в какую компанию вы бы не пришли в 99% случаев будет использоваться система контроля версий Git в не зависимости от используемого языка программирования

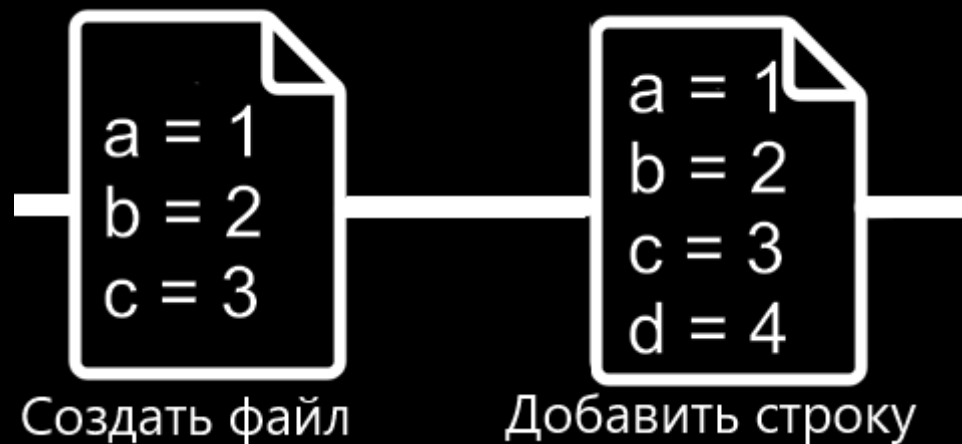


Что можно делать в Git?



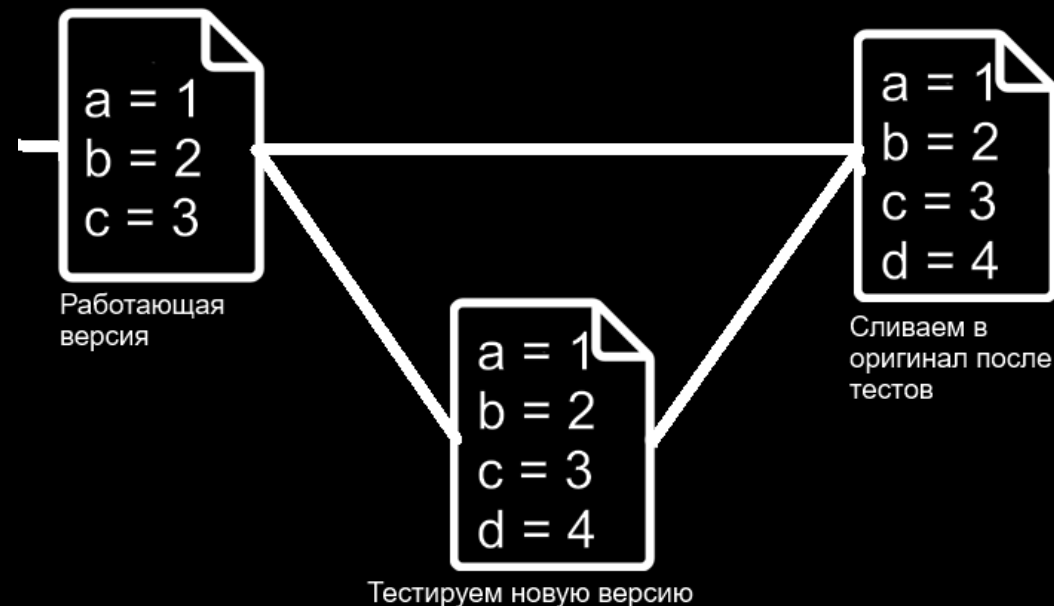
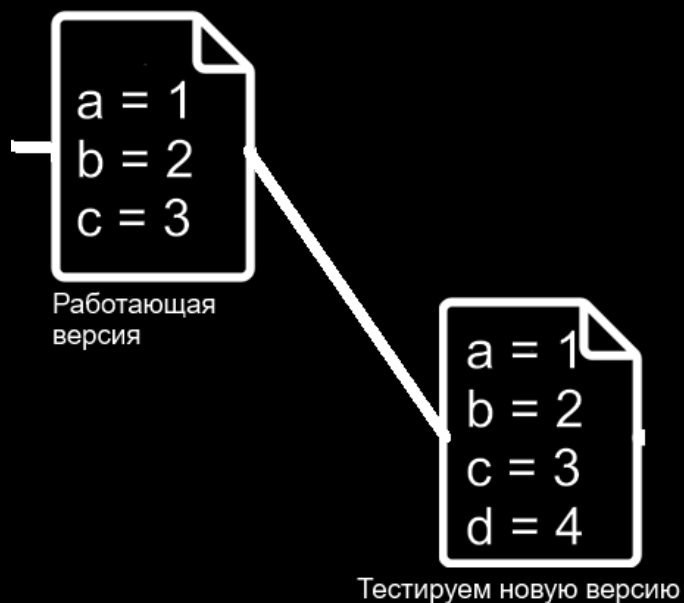
- Отслеживать изменения, например: создали файл, добавили в него код, удалили из него код и т.д.

Что можно делать в Git?



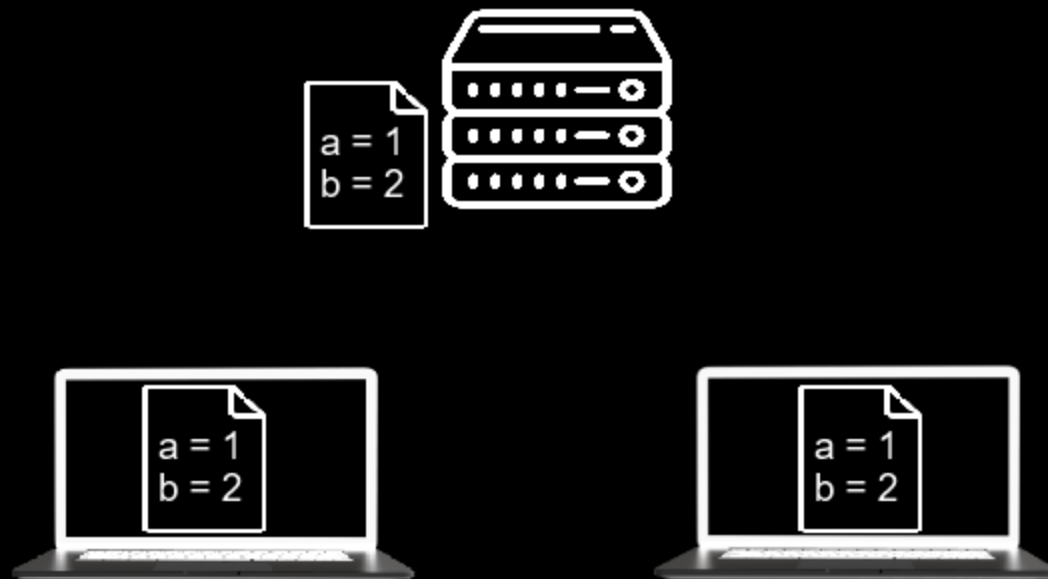
- Возможность откатиться (revert) к последней работающей версии если что-то пошло не так

Что можно делать в Git?



- Протестировать изменения в коде не изменяя работающий оригинал. Создаем мини копию с оригиналом, модифицируем ее как хотим, тестируем по полной и если ничего не отвалилось сливаем (мержим, соединяем) в оригинал

Что можно делать в Git?



- Синхронизировать изменения между разными людьми или устройствами, где последняя актуальная версия кода с его историей хранится на некотором сервере

Git не следует путать с GitHub и прочими

Git — это инструмент, позволяющий реализовать распределённую систему контроля версий.

GitHub — крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки. Веб-сервис основан на системе контроля версий Git. Git-репозиторий, загруженный на GitHub, доступен с помощью разных интерфейсов (командная строка, web и пр. графические интерфейсы).

Кроме GitHub есть другие сервисы, которые используют Git, — например, Bitbucket, Gitlab и некоторые другие. Каждый из них под капотом содержит систему Git, но отличаются они все веб интерфейсами, а также каждый из них привносит свои фичи при работе с проектами.

Мы будем работать именно с GitHub - т.к. это доступный для всех начинающих и опытных разработчиков сервис (ну и конечно один из самых популярных).



Getting started

Для использования git нам необходимо установить его

А также завести аккаунт на GitHub (github.com)

Если при регистрации возникают проблемы, то стоит попробовать указать gmail почту.

И если работаете из консоли - залогиниться:

```
$ git config --global user.name "User Name"
```

```
$ git config --global user.email username@example.com
```

git - утилита и она принимает множество различных флагов на вход, чтобы вспомнить можно воспользоваться --help для просмотра возможностей, например

```
$ git --help
```

```
$ git config --help
```

В VSCode дополнительно помощник с git - расширение GitLens

Создание Репозитория

Репозиторий (от англ. repository — хранилище) — место, где хранятся и поддерживаются какие-либо данные. Иногда можно встретить сокращенно "repo", "git repo". А еще иногда можно услышать "репозитАрий"

Простыми словами это подобно папке с проектом, хранящей не только файлы, но и их историю изменений.

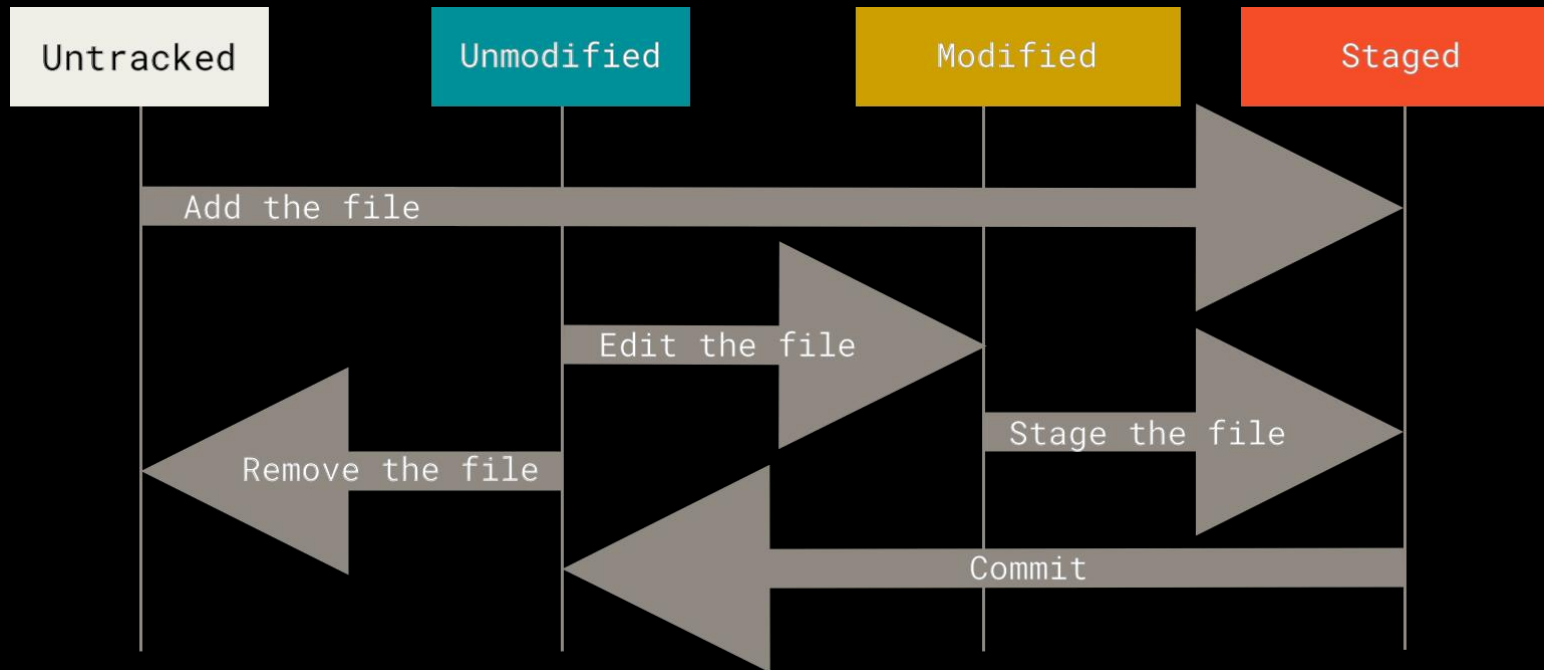
Заходим в github.com, логинимся, а далее вспоминаем как это показывали на лекции, либо гуглим, либо читаем Github doc создание репо

Если проект пустой, то необходимо инициализировать git командой

```
$ git init
```

Команды по инициализации репо обычно показываются github когда заходим в наш пустой репо

Запись изменений в репозиторий (Коммит)



<https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository>

Изменение файлов

Определить состояние файлов можно командой `$ git status`

или в VSCode во вкладке Source Control все что в Changes - измененные, а конкретнее по букве напротив файла M = modified, а U - untracked.

При изменении отслеживаемого файла - он переходит в состоянии измененный и все эти изменения можно увидеть через команду `$ git diff`

или в VSCode во вкладке Source Control открыв нужный файл

"+" и/или зеленый цвет == Добавлено новое

"-" и/или красный цвет == Удалено старое

Следующий этап после модификации файлов – фиксация (индексация) их изменений, делается с помощью команды `$ git add`

В VSCode сделать это можно в Source Control выделив нужные файлы -> нажать ПКМ -> Stage Changes.

.gitignore

Часто можно встретить в корне проекта файл .gitignore. В нем прописываются пути и файлы, состояния которых полностью игнорируются (работает только для неотслеживаемых файлов).

Примеры

```
$ cat .gitignore
```

```
foo/*
```

```
.vscode
```

```
build/*
```

<https://git-scm.com/docs/gitignore>

КОММИТ

Коммит (англ. Commit - фиксировать) - это способ сохранения изменений в коде. Каждый такой коммит содержит информацию о том, что было изменено в коде, кем были внесены эти изменения и когда. Под коммитом можно понимать некоторый снимок состояния проекта.

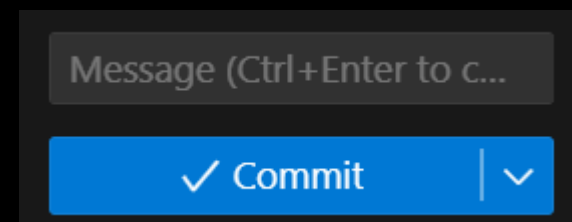
Важно: каждый коммит помимо инфы несет свой уникальный id из 40 символов, в составе могут быть как буквы, так и цифры. - это называется хэш коммита и по нему часто разработчики понимают версию запускаемого кода. Хэш коммита генерится с помощью крипто хэш-функции SHA-1

Теперь когда мы перенесли наши файлы в состояние staged, мы можем их закоммитить, для этого используется команда

```
$ git commit -m "Сообщение коммита: что поменялось"
```

в VSCode - прописывается сообщение над БОЛЬШОЙ синей кнопкой Commit и нажимается эта самая кнопка.

```
$ git commit -am "hello git"
```



КОММИТ



Как теперь просмотреть коммиты?

```
$ git log
```

В верхушке стека будут самые последние коммиты. Интересные флаги для `git log` также будут `--pretty` (`oneline`, `short`, `full`, `fuller` и `format`):

```
$ git log --oneline --graph --decorate
```

- `--oneline` — показывает каждый коммит в одной строке. Кроме того, она показывает лишь префикс ID коммита.
- `--decorate` — печатает все относительные имена показанных коммитов.
- `--graph` - ветки визуализирует красивенько

В VSCode это будет в Source Control Graph. Можно мышкой навести на любой коммит и увидеть все тоже самое.

```
$ git config --global alias.lol 'log --oneline --graph --all'
```

```
$ git lol
```


Удаленные репозитории (push, clone, fetch, pull)

Основы Git - Работа с удалёнными репозиториями

Все наши коммиты на самом деле являются локальными (local), сам наш репозиторий наоборот - удаленный (remote), т.е. он хранится на некотором сервере в интернетах.

От части это полезно тем, что вы можете набрать n-ое кол-во коммитов локально и разом их запустить. А пока их копите у себя локально иногда бывает надо откатиться назад, что-то отменить

Не торопитесь пушить сразу на удаленный сервер, стоит перепроверить ничего ли не забыли

```
$ git push
```

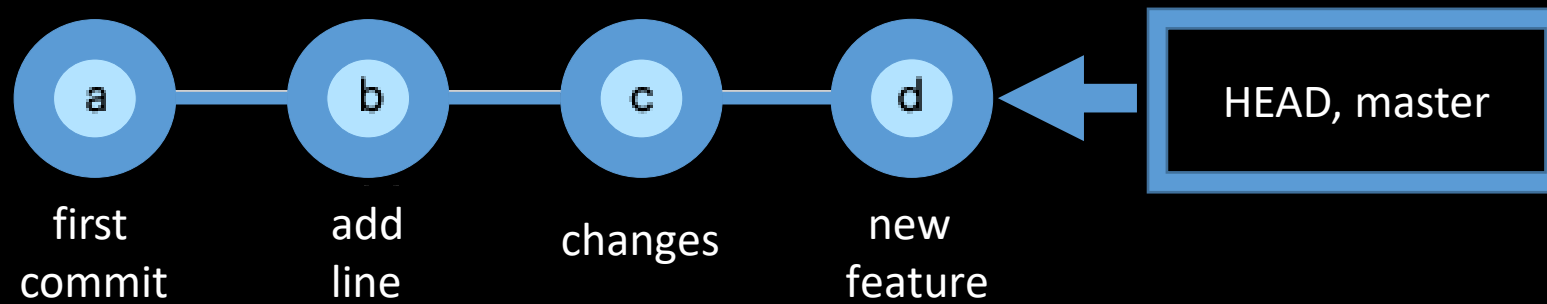
```
$ git pull
```

```
$ git fetch
```

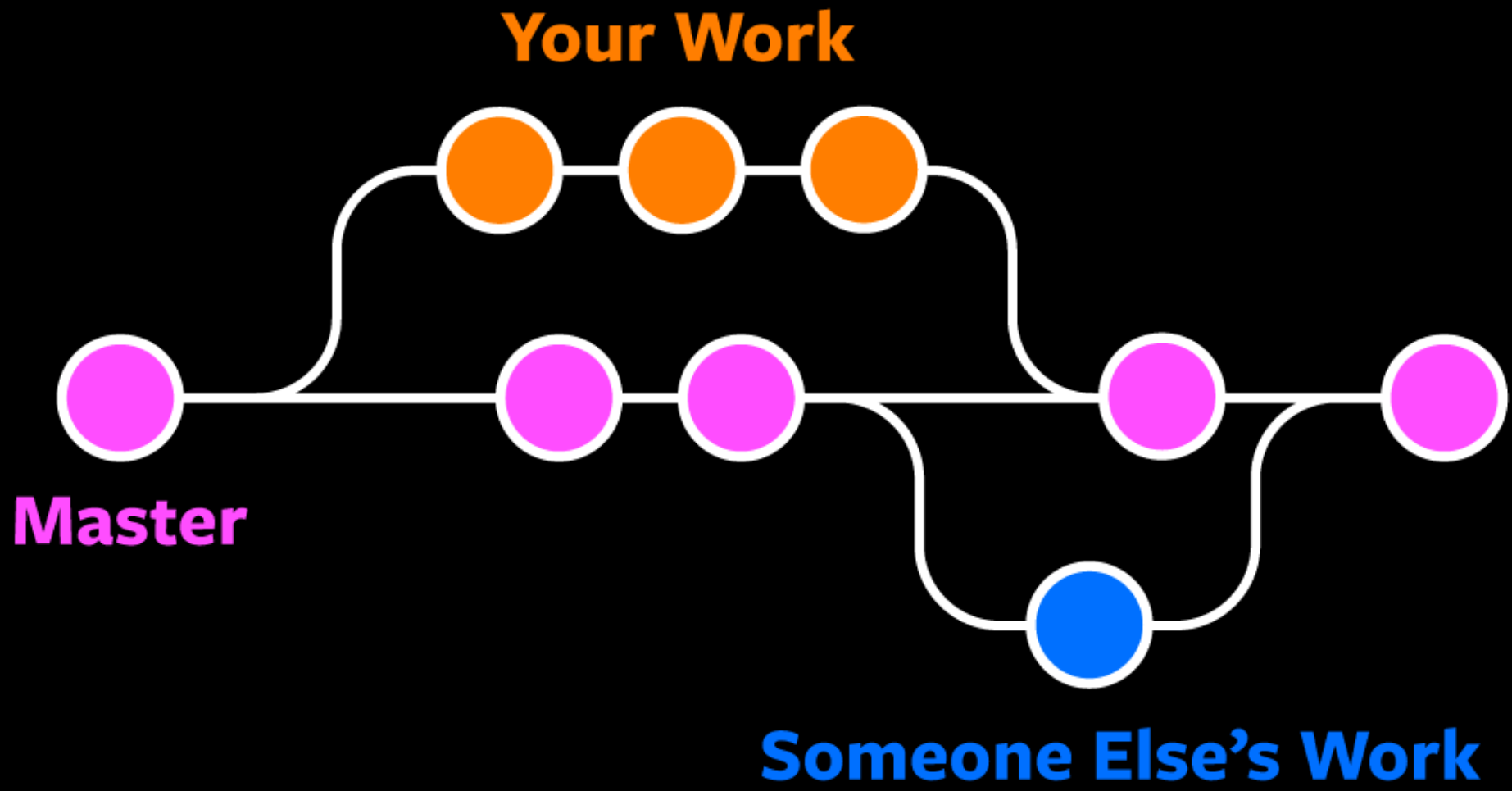
```
$ git clone
```

```
$ git remote -v
```

По умолчанию появится слово origin - алиас (alias - псевдоним) на нашей системе, под которым подразумевается удаленный репозиторий



HEAD - ссылка (указатель) на ваш последний коммит вашей ветки. Можно глянуть `cat .git/HEAD`
Если указывается не ветка, а коммит в этом файле, то это значит у вас detached HEAD == вы пошли бродить по предыдущим коммитам ветки.



Ветвление

- Ветка (англ. Branch) в Git — это простой перемещаемый указатель на один из коммитов.
- По умолчанию, имя основной ветки в Git — master.
- Ветки в Git одна из лучших фиш и они довольно легковесны, т.е. их легко и быстро создать, смиржить и т.д. в отличии от других систем контроля версий
- Полезные команды для веток:

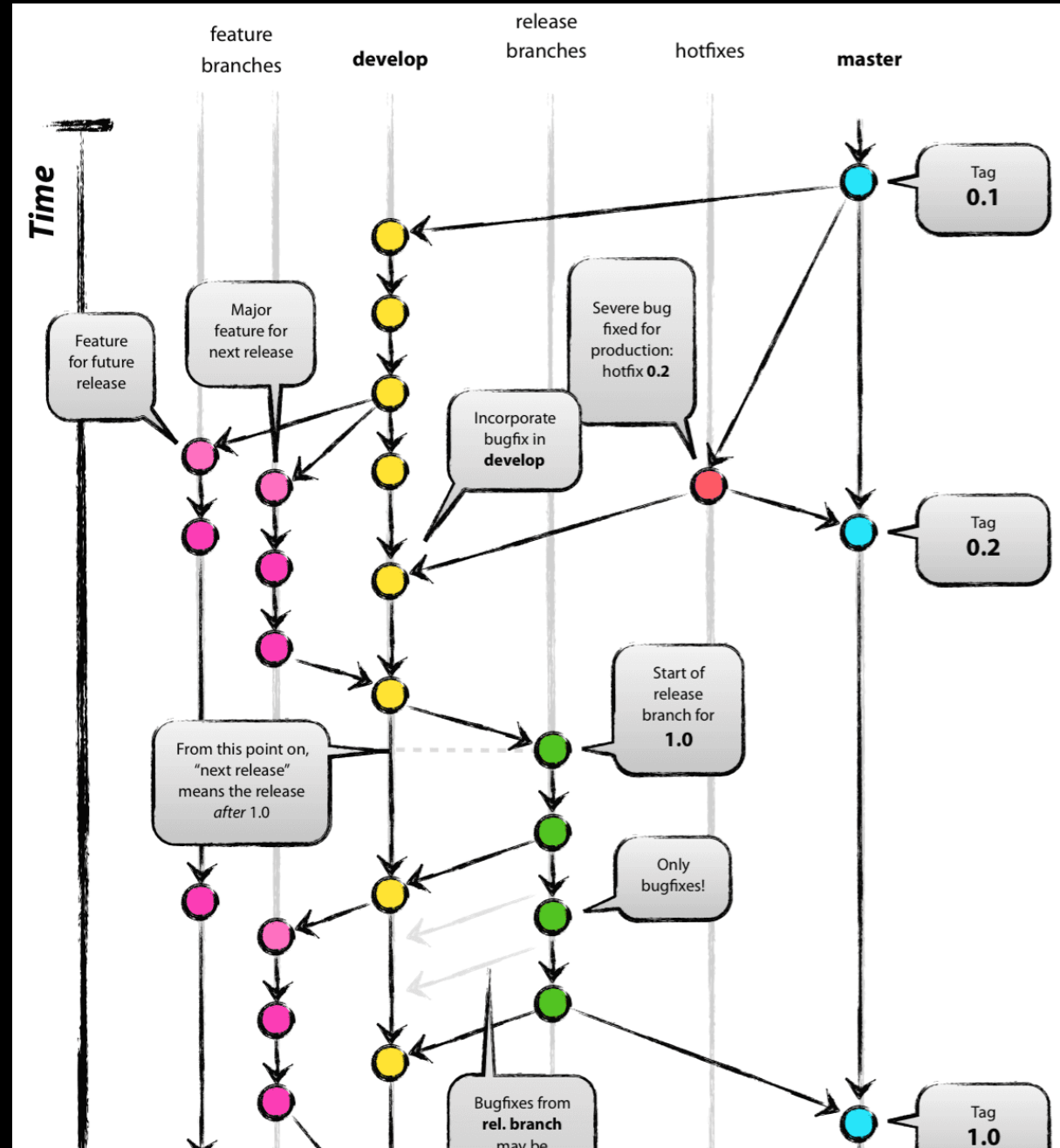
git switch	
git switch -c	
git branch	git diff
git branch <branch-name>	git diff <branchA> <branchB>
git branch -v	git checkout
git branch -d <branch-name>	git checkout -b

<https://git-scm.com/book/en/v2/Git-Branching-Branched-in-a-Nutshell>

<https://git-scm.com/book/en/v2/Git-Branching-Basic-Branched-and-Merging>

В реальной большой разработке обычно есть как минимум ветка master и develop.

В master содержатся самые стабильные версии и релизы, которые отдаются в продакшн. develop - основная ветка для разработки, в ней кипит вся разработка и мержаются все мелкие ветки с новыми фичами, затем проверяется и мержится в master.



Merge

Мердж (слияние, merge) — объединение двух или более веток. В процессе мерджа изменения с указанной ветки переносятся (копируются) в текущую.

Если коммит сливается с тем, до которого можно добраться, двигаясь по истории вперёд, Git упрощает слияние, просто перенося указатель ветки вперёд, потому что в этом случае нет никаких разнонаправленных изменений, которые нужно было бы свести воедино. Это называется «fast-forward» (ff).

Кроме ff, можно увидеть еще:

--ff — (по умолчанию) попытаться выполнить перемотку (fast forward), если не удалось, то создать коммит слияния.

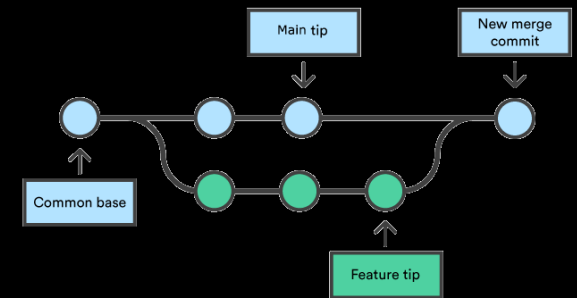
--no-ff — всегда создавать коммит слияния.

--ff-only — попытаться выполнить перемотку, если не удалось, завершить ошибкой.

- Команды для слияния (merge):

git merge

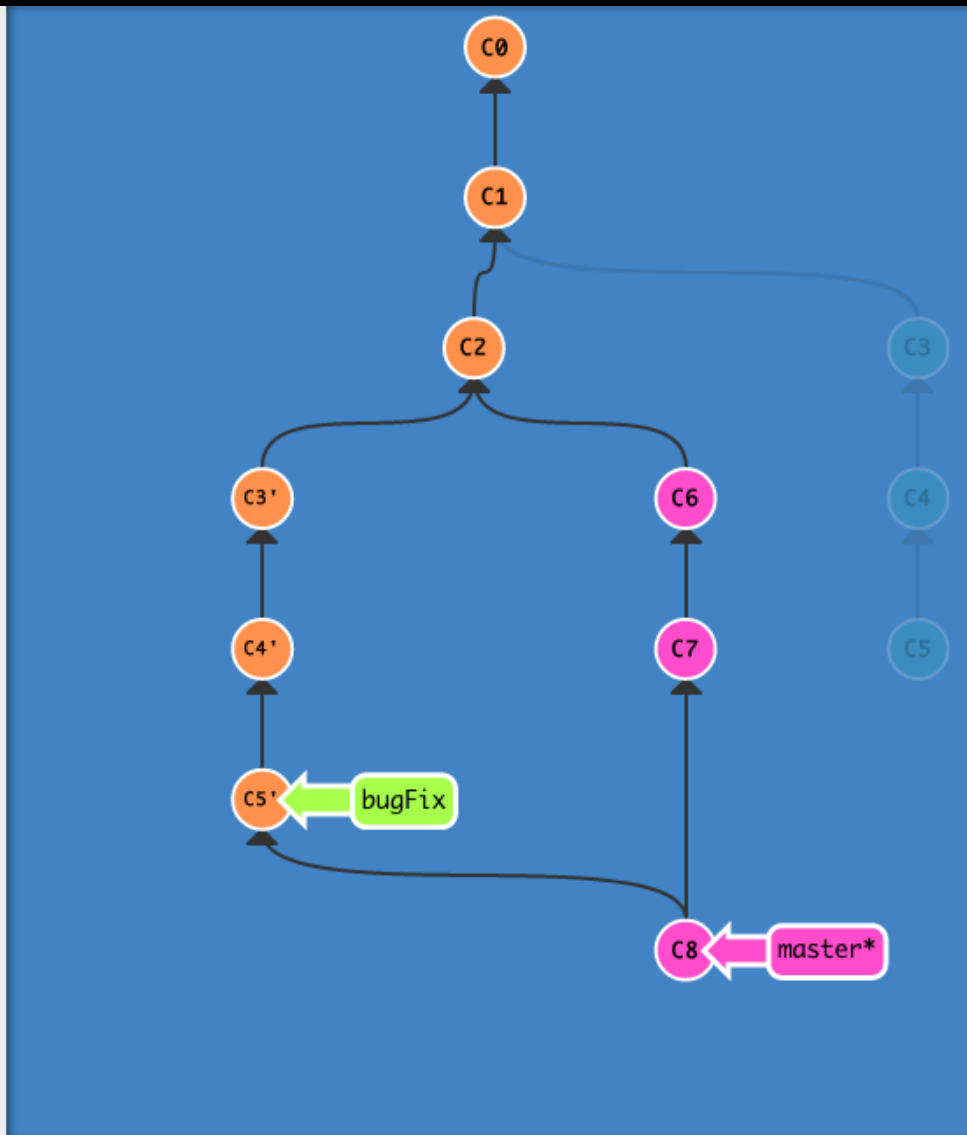
git rebase



<https://git-scm.com/docs/git-merge>

<https://git-scm.com/book/ru/v2/Ветвление-в-Git-Перебазирование>

```
Learn Git Branching
$ gc
$ git checkout HEAD~1
$ git commit
Warning!! Detached HEAD state
$ git checkout -b bugFix
$ gc
$ gc
$ git rebase master
$ git checkout master
$ gc
$ gc
$ git merge bugFix
```



Есть интерактивный сайт, что поможет лучше понять работу веток в Git: [LearnGitBranching](https://learngitbranching.com/)

Отмена изменений

```
$ git commit --amend  
$ git commit --amend -m "New commit message"
```

```
$ git restore --staged *файлы*  
$ git reset HEAD *файлы*
```

```
$ git revert HEAD # или git revert *commit_hash*
```

```
$ git reset --soft HEAD~1  
$ git reset --hard
```

Удаленный репо

```
$ git push -f # или --force, опасненько
```

Последняя надежда:

```
$ git reflog
```

Команда позволяющая слить к себе определённый коммит

```
$ git cherry-pick *<commit_sha>*
```

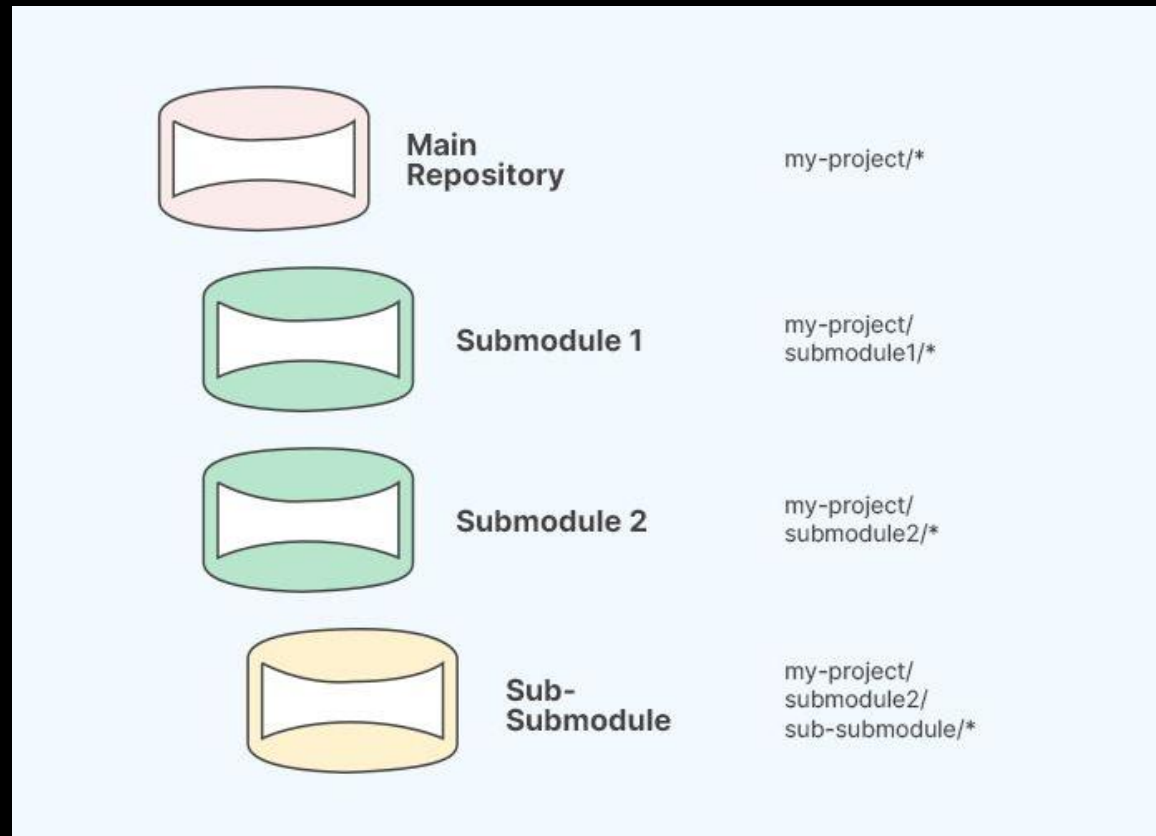
<https://ohshitgit.com/>



Git version in binary

```
git rev-parse --abbrev-ref HEAD  
git log -1 --format=%h  
git log -1 --format=%cd
```

Подмодули



<https://git-scm.com/docs/git-submodule>

<https://git-scm.com/book/en/v2/Git-Tools-Submodules>

Stash (тайник)

`git stash`

`git stash pop`



GitLens

VSCode в Github.com

Откройте свой репозиторий, например
<https://github.dev/kruffka/C-Programming>
и вместо .com введите .dev

Gitlab

А теперь рассмотрим Gitlab и красивые графы коммитов из реальных проектов..



<https://github.com/kruffka/C-Programming>